Dal prototipo all'esecuzione attiva

Requisiti tecnici e percorso di validazione per PokerStars

Repository: poker_reader_stars **Data:** 3 novembre 2026 **Stato:** aggiornato dopo la pulizia della repo; nessuna attivazione eseguita.

1. Obiettivo e requisito centrale

Per passare dall'esperimento a un sistema attivo serve un ciclo completo e affidabile: acquisire lo stato del tavolo, ricostruire la mano, scegliere un'azione valida, eseguirla una sola volta e verificarne l'esito. Collegare il mouse al lettore non è sufficiente.

Il completamento tecnico va distinto dall'impiego consentito. Le regole pubbliche di PokerStars vietano bot e assistenza decisionale in tempo reale [1]. Un eventuale test attivo sulla piattaforma richiederebbe un accordo esplicito con l'operatore, che definisca ambiente e attività consentite. Questo documento non presuppone che tale autorizzazione esista o sia ottenibile.

2. Cosa esiste oggi

- **Acquisizione:** mac_capture.py e native/WindowStream.swift catturano una finestra macOS identificata, con verifiche di identità e disponibilità.
- **Riconoscimento:** stars_reader.py legge carte, piatto, stack, puntate e indicazioni di turno da un tema e layout calibrati, usando template e OCR.
- **Memoria temporale:** hand_tracker.py deduce eventi dagli snapshot; la ricostruzione è parziale e contiene errori riprodotti durante l'analisi.
- **Risultati e archivio:** hand_results.py conferma messaggi visibili nella chat; poker_store.py registra osservazioni ed eventi in SQLite, affiancato dai log JSONL.
- **Anteprime:** dry_run.py produce anteprime senza eseguire azioni. I riferimenti visivi dei controlli sono conservati in docs/coordinate_reference/.

Le precedenti demo locali, basate su scenari e sequenze predefinite, sono state rimosse insieme al materiale editoriale sul bet sizing e all'helper Swift inutilizzato. Non esiste un esecutore attivo nel nucleo conservato. I pagamenti osservati nella chat non costituiscono una contabilità completa delle vincite o delle perdite.

3. Evidenze già disponibili

La suite mantenuta comprende 42 test Python del nucleo, tutti superati nell'analisi. È passata anche una prova da immagine tramite CLI fino alla registrazione in SQLite. I 20 test delle vecchie demo appartenevano ai prototipi rimossi e non fanno più parte della repo.

Questi risultati non validano il funzionamento live, il mouse su PokerStars, altri temi grafici o la qualità economica di una strategia. I difetti descritti nella pagina seguente sono stati riprodotti con sequenze sintetiche aggiuntive, non coperte dalla suite esistente.

Prima priorità: rendere affidabile lo stato

4. Correzioni necessarie nel tracker

P0 - False nuove mani causate da carte incerte. La sequenza Qc 6h -> ?c 6h -> Qc 6h genera tre identificativi di mano. La stabilità attualmente modifica la confidenza, ma non impedisce le transizioni. Separare lo stato candidato da quello confermato e impedire che carte sconosciute ridefiniscano la mano. Riferimenti: hand_tracker.py:95 e hand_tracker.py:141.

P0 - Mani diverse fuse dopo un intervallo senza carte. La sequenza preflop Qc 6h -> nessuna carta -> As Ad conserva lo stesso identificativo. Mantenere memoria dell'ultima mano confermata e combinare distribuzione, dealer, puntate e altri segnali disponibili; quando l'evidenza manca, segnalare il confine come incerto. Riferimento: hand_tracker.py:141.

P0 - Puntata persa al cambio di street. Con 100 centesimi preflop e 40 già visibili al flop, viene registrato soltanto il cambio di street: il confronto usa ancora le puntate del frame precedente. Confrontare le nuove contribuzioni con una base coerente con la nuova street e gestire le animazioni di raccolta delle fiches. Riferimento: hand_tracker.py:180.

P0 - Rilancio e chiamata attribuiti al posto errato. Gli aumenti vengono ordinati per numero di posto. Se il posto 6 rilancia e il posto 1 chiama tra due frame, le azioni possono risultare invertite. Usare l'ordine di azione e la cronologia disponibile; se la sequenza non è ricostruibile, conservare l'ambiguità senza inventare un aggressore. Riferimento: hand_tracker.py:201.

P0 - Importo da chiamare falsamente uguale a zero. Una puntata avversaria illeggibile viene esclusa dal massimo. hero_to_call_cents può così risultare zero. Propagare l'incertezza agli importi derivati e impedire una decisione che dipenda da dati mancanti. Riferimento: hand_tracker.py:125.

5. Criteri di completamento della prima fase

- Ogni difetto sopra ha un test di regressione che fallisce prima della correzione e passa dopo.
- Le letture incerte non modificano silenziosamente identità della mano, aggressore o importi confermati.
- Un'interruzione della cattura invalida la continuità necessaria alla conferma; il recupero richiede nuove evidenze.
- I log distinguono osservazioni, inferenze, eventi confermati e dati sconosciuti.
- I replay vengono confrontati con annotazioni indipendenti, includendo frame di transizione e non soltanto immagini statiche favorevoli.

P1 - Riproducibilità: il README è stato ripristinato e le demo con i launcher errati sono state rimosse. Restano da includere in Git i moduli necessari e predisporre fixture distribuibili. Le dipendenze e i comandi di avvio sono ora documentati nel README; le immagini private di regressione restano locali.

Componenti da integrare per l'esecuzione

6. Motore decisionale

Definire un'interfaccia che riceva lo stato confermato e restituisca azione, importo totale richiesto, versione dello stato e motivazione verificabile. Gli importi restano in centesimi interi; rilancio totale e incremento devono avere significati distinti.

Il motore deve conoscere le azioni ammesse e poter restituire un'astensione quando mancano informazioni essenziali. Valutare la strategia offline e sulla replica: l'assenza di errori software non dimostra un vantaggio economico, e questo documento non propone una strategia già validata.

7. Lettura dei controlli e dei vincoli

Distinguere i pulsanti effettivamente presenti: check, call, bet, raise e fold. Leggere costo della chiamata, limiti del rilancio, stack disponibile e valore del campo importo. Gestire pulsanti disabilitati, all-in, sit-out e schermate intermedie.

Le regioni attuali di dry_run.py sono riferimenti visivi nell'immagine canonica; non sono coordinate desktop utilizzabili direttamente. Un adattatore deve gestire geometria della finestra, scala Retina e variazioni del layout, verificando nuovamente il bersaglio al momento dell'azione.

8. Esecutore e controllo della sequenza

Separare la proposta della strategia dall'esecuzione. Una sequenza proposta è: osservazione, validazione, preparazione, nuova verifica, invio singolo, conferma oppure arresto.

- Prima dell'invio, verificare finestra, mano, street, turno, freschezza del frame e compatibilità dell'importo con i controlli visibili.
- Invalidare la proposta se lo stato cambia durante la preparazione.
- Associare un identificativo univoco al tentativo ed evitare invii concorrenti o duplicati.
- Verificare il valore inserito prima della conferma dell'azione.
- Prevedere arresto manuale e sospensione automatica quando finestra, layout o stato non sono verificabili.

L'esecutore deve essere implementato come componente separato: nella repo ripulita restano le sole anteprime. Il passaggio a un'interfaccia esterna richiede prove dedicate nell'ambiente autorizzato.

9. Conferma e registrazione dell'esito

L'invio di un click non dimostra che l'azione sia stata accettata. Confrontare lo stato successivo con l'esito previsto, usando turno, puntata, stack e altri segnali disponibili. Prevedere un tempo massimo di conferma e distinguere esito confermato, rifiutato e sconosciuto.

In caso di esito sconosciuto, sospendere l'esecuzione senza ripetere automaticamente il click. Registrare identificativo della mano e del tentativo, snapshot prima e dopo, decisione, importo, tempi, conferma ed eventuale causa di arresto. Anche dopo un riavvio, un tentativo rimasto incerto richiede riconciliazione prima di riprendere.

Validazione e condizioni di rilascio

10. Percorso proposto

Fase A - Correzioni e replay offline. Risolvere i difetti del tracker, consolidare le fixture e confrontare gli eventi ricostruiti con sequenze annotate. Uscita: regressioni risolte e limiti di riconoscimento documentati.

Fase B - Ciclo completo in un ambiente di prova. Predisporre un banco di test dedicato e collegare stato, strategia, controlli, esecutore e conferma. Provare importi invalidi, azioni duplicate, turni scaduti e spostamenti della finestra. Uscita: ogni tentativo ha un esito tracciato; nei casi ambigui il sistema si arresta correttamente.

Fase C - Osservazione in un ambiente consentito. Misurare qualità delle letture, frequenza di ambiguità e latenza dell'intera elaborazione senza inviare azioni. Le autorizzazioni devono coprire anche l'acquisizione e il trattamento dei dati previsti. Uscita: risultati misurati rispetto a soglie definite prima della prova.

Fase D - Test attivo concordato con l'operatore. Solo se disponibile un'autorizzazione esplicita, usare l'ambiente, gli account e i limiti concordati. Uscita: evidenze end-to-end, arresto e recupero verificati, tracciabilità completa e criteri di accettazione soddisfatti.

11. Matrice minima di prova

- **Mani e turni:** fine preflop, nuova distribuzione, cambio di street, più azioni tra due frame e passaggio dal posto 6 al posto 1.
- **Importi:** puntate illeggibili, stack insufficiente, rilanci minimi, all-in, side pot e distinzione tra importo totale e incremento.
- **Interfaccia:** ridimensionamento, scala diversa, controlli coperti o disabilitati, cambio scheda e finestra chiusa.
- **Tempi e guasti:** frame obsoleto, timeout, disconnessione, arresto durante la preparazione, riavvio dopo un invio non confermato.
- **Tracciabilità:** corrispondenza tra decisione, tentativo, conferma, JSONL e SQLite; nessuna duplicazione del tentativo.

12. Quando considerare concluso l'esperimento

Il traguardo tecnico è una sessione completa nell'ambiente di prova con decisioni, esecuzione e risultati verificati automaticamente, inclusi i casi di incertezza. La prontezza per un ambiente esterno richiede inoltre dati indipendenti, soglie misurabili di accuratezza e latenza, procedure di arresto e recupero e un'installazione riproducibile.

Le soglie quantitative e la durata minima delle prove sono ancora da definire: non sono stati raccolti dati sufficienti per dichiarare il sistema pronto alla produzione. L'autorizzazione della piattaforma rimane una condizione distinta dalla validazione tecnica.

Fonti e limiti del documento

[1] PokerStars, **Prohibited Programs, Tools and Services**, consultato il 3 novembre 2026: pokerstars.com/poker/room/prohibited/

[2] Analisi del working tree locale e verifiche eseguite nella conversazione: stars_reader.py, hand_tracker.py, hand_results.py, poker_store.py, mac_capture.py, dry_run.py, native/ e tests/.

[3] docs/validation.md: storico delle prove visive, limiti delle calibrazioni e distinzione tra osservazioni della chat e storico completo delle mani. I riferimenti alle righe di codice corrispondono allo stato analizzato e possono cambiare con le modifiche.